

Python and R for Biologists

DATA405 Genomic Data Science • Lecture 10 • Sep 28, 2026

ATGCCGGAGGCCCCGCCTCTGCTGTTGGCAGCTGTGTTGCTGGGCCTGGTGCTGCTGGTGGTG

Jinrui Xu, Department of Biology, Howard University

Built on: Python for Biologists (Jones) • Software Carpentry • R for Data Science (Wickham et al.) • Hands-On Programming with R (Grolemund)

Why a biologist learns to program (Jones, ch. 1)

- **Biology is now a data science.** A sequencing run yields millions of short text strings; a microscope yields gigabytes of images. Spreadsheets stop at a million rows.
- **Your question is new.** Tools exist for common steps (mapping, counting). They do not exist for the specific question in your proposal.
- **Automation and reproducibility.** A script is a protocol that runs itself, identically, on the next 500 samples.
- **Reading.** Methods sections, GitHub repositories and Bioconductor vignettes are written in code.

Goal: after two hours you can read a short Python or R script and write a few lines yourself.

Why Python AND R?

- **Python** (van Rossum, 1991): general purpose, reads like English, the language of pipelines, web services and machine learning. Biopython since 1999.
- **R** (Ihaka & Gentleman, 1993; from Bell Labs' S, 1976): built by statisticians; a test, a model and a plot are one line each. Bioconductor since 2001 (DESeq2, edgeR, limma, Seurat).
- Most genomics projects use both: a Python/shell pipeline produces a count table; an R notebook analyses and plots it.
- Learn one well, the other by translation. Today: Python from zero (Jones's order), then R by translation (R4DS's order).

Today

	Minutes	What you will be able to do	Following
Part A · Python from zero	70	text, files, lists & loops, conditions, functions, dictionaries, regular expressions; pandas & a plot	Jones ch. 2–8; Software Carpentry
Part B · R by translation	40	vectors, strings, data frames, base-R plots, one-line statistics	R4DS ch. 1–4, 12; HOPR ch. 1–5
Part C · Both	10	when to use which; common bugs; using ChatGPT; the lab	

Every code slide shows its output. Predict before I run.

How to learn a language (Software Carpentry)

- Type the code yourself; do not paste. Fingers remember.
- Change one thing and re-run. Predict the output first; then check.
- Errors are information, not failure. Read the last line of the message.
- Test on a case whose answer you already know before trusting a result on new data.
- Keep a notebook: the code, its output and your comment in one file (Jupyter, R Markdown/Quarto).

Part A

Python from zero

Source structure: M. Jones, Python for Biologists (2013; pythonforbiologists.com) · Software Carpentry, Programming with Python

A1 • Printing and manipulating text (Jones ch. 2)

Python

```
print("Hello, genome")  
# → Hello, genome  
  
my_dna = "ATGCGTA"  
print(my_dna)           # → ATGCGTA  
print(len(my_dna))      # → 7    the length  
print(my_dna.lower())   # → atgcgta  
print(my_dna + "TGA")   # → ATGCGTATGA    concatenation  
  
# comments start with #; Python ignores them
```

Text lives in quotes and is called a string. `print()` shows a value; `len()` measures it; a method like `.lower()` transforms a copy.

Variables

Python

```
my_dna = "ATGCGTA"      # store a value under a name
length = len(my_dna)    # a number (an integer)
gc = 0.43                # a decimal (a float)

print(my_dna, length, gc)
# → ATGCGTA 7 0.43

length = length + 3      # replace the old value
print(length)            # → 10

type(my_dna), type(length), type(gc)
# → str, int, float
```

= means 'becomes', not 'equals'. A variable is a labelled box you can reuse and overwrite. Names: letters, digits, underscores; case matters.

Jones's first program: AT content

Python

```
my_dna = "ACTGATCGATTACGTATAGTATTTGCTATCATACATATATATCGATGCGTTTCAT"

length = len(my_dna)
a_count = my_dna.count("A")
t_count = my_dna.count("T")
at_content = (a_count + t_count) / length

print("AT content is " + str(at_content))
# → AT content is 0.6851851851851852

print(f"AT content is {at_content:.2f}")    # formatted: 2 decimals
# → AT content is 0.69
```

.count() counts a substring. **str()** turns a number into text so it can be joined; an f-string does the same more readably.

Complementing DNA with replace() — and a trap

Python

```
my_dna = "ACTGATCGATTACGTATAGTATTTGCTATCATACATATATATCGATGCGTTTCAT"

# naive: replace each base one at a time
s = my_dna.replace("A", "T")      # A → T    ... but now every T is an old A!
s = s.replace("T", "A")          # turns them ALL into A
print(s[:10])                    # → ACAGAACGAA    wrong

# Jones's fix: go through lower case as a temporary alphabet
s = my_dna.replace("A", "t").replace("T", "a")
s = s.replace("C", "g").replace("G", "c")
print(s.upper()[:10])            # → TGACTAGCTA    correct
```

A program does exactly what you say. The first version has no error message and a wrong answer — the worst kind of bug. Test on 'ATGC' → 'TACG'.

Indexing and slicing: the first letter is 0

Python

```
dna = "ATGCCGGAGGCC"
```

```
#      0123456789...
```

```
dna[0]      # → 'A'    first letter
```

```
dna[-1]     # → 'C'    last letter
```

```
dna[0:3]    # → 'ATG'  letters 0,1,2 – 3 is NOT included
```

```
dna[3:6]    # → 'CCG'  the second codon
```

```
dna[:3]     # → 'ATG'  from the start
```

```
dna[9:]     # → 'GCC'  to the end
```

```
# find a restriction site (EcoRI = GAATTC)
```

```
seq = "ACTGAATTCCGTA"
```

```
seq.find("GAATTC")    # → 3    position of the first letter (0-based)
```

Python counts from 0 and a slice [a:b] stops before b. GTF, VCF and SAM files count from 1: subtract one when crossing over.

Predict (Jones ch. 2 exercise)

YOUR TURN EcoRI cuts GAATTC between G and A. For seq = "ACTGAATTCCGTA", what are the two fragment lengths?

- A. 3 and 10
- B. 4 and 9
- C. 5 and 8
- D. 6 and 7

ANSWER B. find() gives 3; the cut is after the G, so the first fragment is seq[:4] (4 letters) and the second seq[4:] (9 letters). Off-by-one is the lesson.

A2 • Reading and writing files (Jones ch. 3)

Python

```
# read a whole file as one string
text = open("data/COMT_cds.fa").read()
print(text[:60])

# read line by line – the usual way
for line in open("data/COMT_cds.fa"):
    print(line.rstrip())          # rstrip() removes the newline

# write a file
out = open("gc.txt", "w")         # "w" = write (overwrites!)
out.write("COMT\t0.618\n")        # \t = tab, \n = new line
out.close()
```

Files are text. `open()` gives you the lines; every line ends with an invisible newline that `rstrip()` removes. Forgetting it is a classic bug.

A3 • Lists and loops (Jones ch. 4)

Python

```
genes = ["COMT", "BCR", "CHEK2", "NF2"]
len(genes)           # → 4
genes[0]             # → 'COMT'
genes[-1]            # → 'NF2'
genes.append("SOX10")
sorted(genes)        # → ['BCR', 'CHEK2', 'COMT', 'NF2', 'SOX10']

for gene in genes:   # the loop body runs once per item
    print(gene, len(gene))

for i in range(3):   # 0, 1, 2
    print(i)
```

A list is an ordered collection in `[ ]`. `for item in collection:` — the indented block is repeated for each item.

Loop + accumulator: the shape of most small programs

Python

```
dna = "ATGCCGGAGGCC"

gc_count = 0                                # start the accumulator
for base in dna:                             # one letter at a time
    if base == "G" or base == "C":
        gc_count = gc_count + 1

print(gc_count / len(dna))                  # → 0.75

# Jones: sort DNA sequences by length
seqs = ["ATGCGT", "AT", "ATGCGTAAACGT", "ATGC"]
sorted(seqs, key=len)
# → ['AT', 'ATGC', 'ATGCGT', 'ATGCGTAAACGT']
```

Trace it by hand for 'ATGC': gc_count $0 \rightarrow 0 \rightarrow 0 \rightarrow 1 \rightarrow 2$; $2/4 = 0.5$. Tracing by hand is how you find bugs.

Reading a FASTA into a list, then a string

FASTA: a header line starting with >, then the sequence over many lines.

Python

```
seq_lines = []                # an empty list
for line in open("data/COMT_cds.fa"):
    if not line.startswith(">"):    # skip the header
        seq_lines.append(line.rstrip())

cds = "".join(seq_lines)      # glue the pieces into one string
len(cds)                      # → 816
cds[:30]                      # → 'ATGCCGGAGGCCCGCCTCTGCTGTTGGCA'
```

Everything here you already know: a list, a loop, a condition, a string method. Programs are small parts combined.

A4 • Conditions (Jones ch. 5)

Python

```
q = 35                # a Phred base-quality score
q >= 30               # → True    (a Boolean)

if q >= 30:
    print("good base")
elif q >= 20:
    print("acceptable")
else:
    print("discard")

# combine: and / or / not
q >= 20 and q < 30    # → False
"ATG" in "CCATGCC"    # → True

# filter a list with a condition
```

```
long_seqs = [s for s in seqs if len(s) > 4]    # list comprehension
```

The colon starts a block; indentation defines it. == tests equality (two signs); = assigns (one sign).

Predict

YOUR TURN `quals = [35, 22, 8, 40]`. How many pass this filter? `[q for q in quals if q >= 20 and q < 40]`

- A. 1
- B. 2
- C. 3
- D. 4

ANSWER B. 35 and 22 pass; 8 fails the first test; 40 fails `q < 40` (strict). Boundary conditions deserve a test case each.

A5 • Writing functions (Jones ch. 6)

Python

```
def gc_content(dna):  
    """Fraction of letters that are G or C."""           # a docstring  
    g = dna.count("G")  
    c = dna.count("C")  
    return (g + c) / len(dna)  
  
gc_content("ATGCCGGAGGCC")    # → 0.75  
gc_content("AAAA")            # → 0.0  
gc_content("GGCC")            # → 1.0  
  
def at_content(dna, decimals=2):          # a default argument  
    at = (dna.count("A") + dna.count("T")) / len(dna)  
    return round(at, decimals)
```

def name(inputs): ... return output. Test a new function on cases with known answers (0.0, 1.0) before real data. Functions make code reusable and testable.

A6 • Regular expressions (Jones ch. 7)

Python

```
import re

dna = "ATCGCGAATTCAC"
re.search("GAATTC", dna)           # a match object (truthy) or None

# variable sites: a restriction enzyme that cuts GG(A or T)CC
if re.search(r"GG(A|T)CC", "ATGGACCTA"):
    print("site found")

# character classes and repeats
re.findall(r"[AT]{6,}", dna)        # runs of 6+ A/T
m = re.search(r"GAATTC", dna)
m.start()                          # → 6   where it starts

# split on every ambiguous base
re.split(r"[^ATGC]", "ACTNGCATRGCTACGT")
```

A regular expression is a pattern language for text: | for alternatives, [] for a set, {n,} for repeats, ^ for 'not'. Biology is full of patterns.

A7 • Dictionaries (Jones ch. 8)

Python

```
# the genetic code IS a dictionary: codon → amino acid
code = {"ATG": "M", "GTG": "V", "GAG": "E", "TAA": "*"}
code["ATG"]          # → 'M'
code["TTT"]          # → KeyError: 'TTT'    (not in the table)
code["TTT"] = "F"    # add an entry
"GTG" in code        # → True

# Jones: count k-mers with a dictionary
counts = {}
for i in range(len(dna) - 2):
    kmer = dna[i:i+3]
    counts[kmer] = counts.get(kmer, 0) + 1
sorted(counts.items(), key=lambda kv: -kv[1])[:3]
```

{key: value}. A dictionary answers 'what does X map to?' in one step. `counts.get(k, 0)` reads a key that may not exist yet.

The whole genetic code, built with three nested loops

Python

```
bases = "TCAG"
aas = ("FFLLSSSSYY**CC*WLLLLPPPPHHQQRRRR"
      "IIIMTTTTNNKKSSRRVVVVAAAADDEEGGGG")
codon_table = {}
i = 0
for a in bases:
    for b in bases:
        for c in bases:
            codon_table[a + b + c] = aas[i]
            i = i + 1

len(codon_table)          # → 64
codon_table["ATG"]        # → 'M'
codon_table["TAA"]        # → '*'
```

The 64-letter string lists amino acids in TTT, TTC, TTA, ... order; the loops walk the same order. Standard code, NCBI table 1.

Putting it together: translate COMT, find rs4680

Python

```
def translate(dna):
    protein = ""
    for i in range(0, len(dna) - 2, 3):          # 0, 3, 6, ...
        protein = protein + codon_table[dna[i:i+3]]
    return protein

protein = translate(cds)
protein[:10]          # → 'MPEAPPLLLA'
len(protein) - 1      # → 271  amino acids (+ stop)

i = (158 - 1) * 3      # codon 158 → index 471
cds[i:i+3]            # → 'GTG'  valine
mutant = cds[:i] + "A" + cds[i+1:]
translate(mutant)[157] # → 'M'      Val158Met (rs4680)
```

Twelve lines reproduce a textbook result: rs4680 turns valine 158 of COMT into methionine. Every line is something from this hour.

Jones exercise: splice out an intron

Python

```
# exon 1 = positions 1–63, intron 64–90, exon 2 = 91–end (1-based, as in GenBank)
seq =
("ATCGATCGATCGATCGACTGACTAGTCATAGCTATGCATGTAGCTACTCGATCGATCGATCGATCGATCGATCGATCGATCGATCATGCTATCATCGATCGATATC
GATGCATCGACTACTAT")
exon1 = seq[0:63]           # 1-based 1..63 → 0-based [0:63]
exon2 = seq[90:]            # 1-based 91.. → 0-based [90:]
coding = exon1 + exon2
len(coding) / len(seq)      # fraction that is coding
```

Every published coordinate is 1-based and inclusive; every Python slice is 0-based and exclusive at the end. Convert once, on paper, then code.

FASTQ: four lines per record (Jones: sampling reads)

Python

```
seqs, quals = [], []
with open("data/reads_5k.fastq") as f:
    while True:
        name = f.readline()           # line 1: @name
        if not name:                  # empty string = end of file
            break
        seqs.append(f.readline().strip()) # line 2: sequence
        f.readline()                  # line 3: '+'
        quals.append(f.readline().strip()) # line 4: quality
len(seqs)                            # → 5000

# Phred+33: quality letter → number
[ord(c) - 33 for c in quals[0][:8]]   # → [35, 35, 35, 35, 35, 40, 40, 40]
```

```
import random
```

```
random.seed(1)
```

A while loop runs until a condition fails — here, until the file ends. `ord()` gives a character's code: 33 is the FASTQ offset.

```
sample = random.sample(range(len(seqs)), 100) # 100 random read indices
```

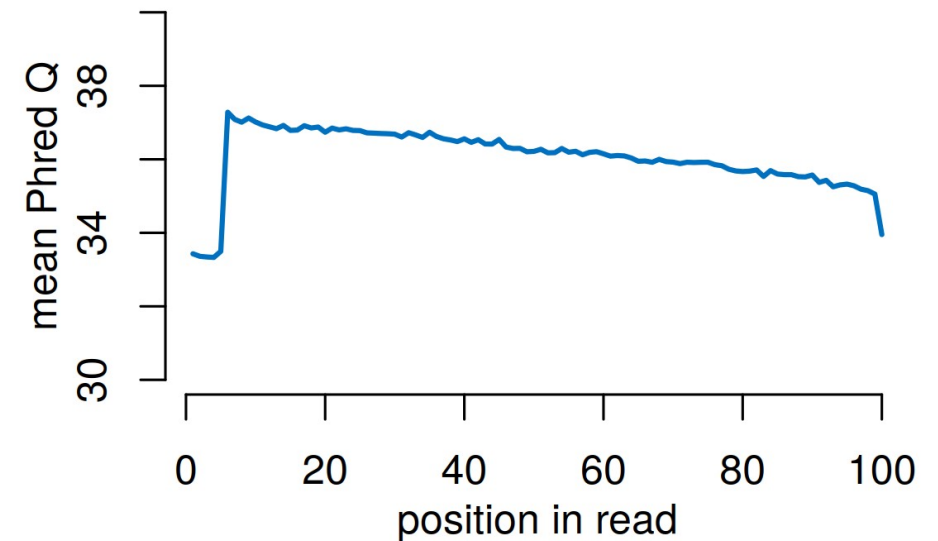
Mean quality per position — the FastQC plot from raw bytes

Python

```
L = len(seqs[0])                # 100
mean_q = []
for i in range(L):
    total = 0
    for q in quals:
        total = total + (ord(q[i]) - 33)
    mean_q.append(total / len(quals))

[round(x, 1) for x in mean_q[::20]] # every 20th position
# → [33.4, 36.8, 36.5, 36.1, 35.7]

fig, ax = plt.subplots(figsize=(4, 2.6))
ax.plot(range(1, L + 1), mean_q)
ax.set_xlabel("position in read"); ax.set_ylabel("mean Phred Q")
```



Nested loops: for each position, for each read. Q = 30 means 1 error in 1,000. Quality drifts down along the read — the classic Illumina profile.

Reading an error message

- **NameError: name 'dan' is not defined** — a typo in a variable name.
- **IndexError: string index out of range** — letter 12 of a 12-letter string (the last is 11).
- **KeyError: 'TTN'** — the codon has an N; it is not in the dictionary.
- **TypeError: can only concatenate str (not "float") to str** — you forgot str() (Jones's AT-content lesson).
- **IndentationError** — a block is not indented consistently.
- Read the last line first, then the line number. The loud error is your friend; the silent wrong answer is the enemy.

Libraries: standing on other people's code

- Python itself is small. Power comes from packages you import: pandas (tables), numpy (numbers), matplotlib (plots), scipy (statistics), Biopython (sequences, files, BLAST).
- **import pandas as pd** — load once, refer to it as pd.
- Install with `pip install pandas` or `conda install pandas` (Anaconda/Miniconda is the biologist's usual route on Windows and Mac).
- Biopython in two lines: `from Bio.Seq import Seq; Seq(cds).translate()` — the same thing you just wrote, tested by thousands of users.

pandas: the table of chr22 genes (Software Carpentry style)

Python

```
import pandas as pd
genes = pd.read_csv("data/chr22_genes.tsv", sep="\t")

genes.shape                # → (1747, 7)
genes.head(3)              # first rows
genes["gene_type"].value_counts().head(3)
# → lncRNA 747 · protein_coding 447 · processed_pseudogene 142

pc = genes[genes["gene_type"] == "protein_coding"]    # filter rows
pc["length_bp"].median()                             # → 23780.0
pc.sort_values("length_bp", ascending=False).head(3)
# → LARGE1 760660 · TTC28 701827 · SYN3 550562
genes.groupby("gene_type")["length_bp"].median()
```

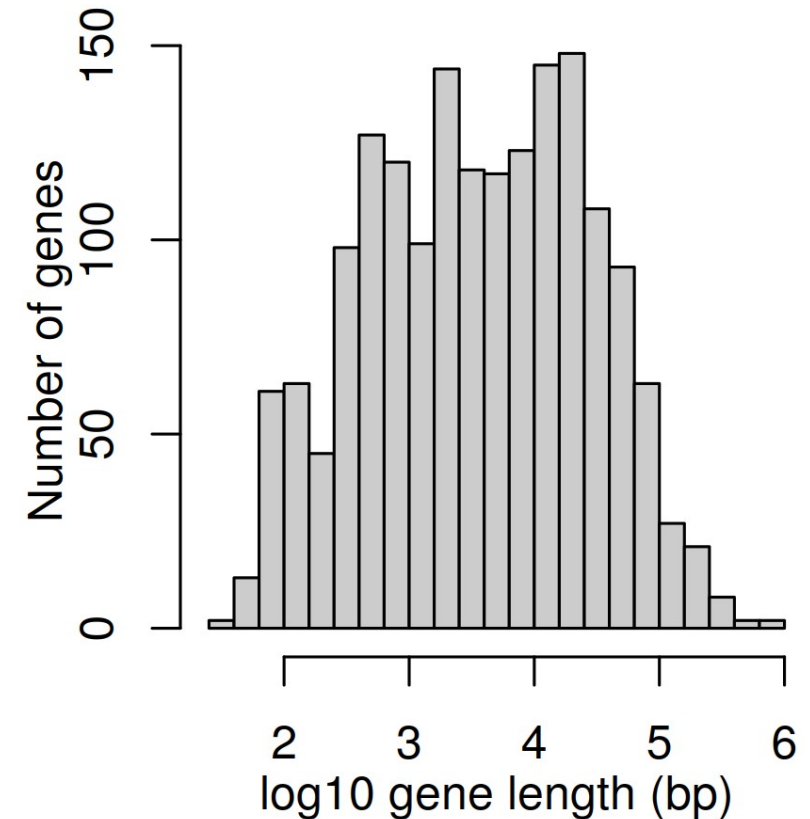
A DataFrame is a table with named columns. The five verbs — read, filter, sort, group/summarize, join (merge) — cover most analysis.

matplotlib: one figure, sized in inches

Python

```
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(3.2, 3.2))    # one
journal column
ax.hist(np.log10(genes["length_bp"]), bins=30,
color="grey")
ax.set_xlabel("log10 gene length (bp)")
ax.set_ylabel("Number of genes")
plt.tight_layout()
fig.savefig("fig1.pdf")
plt.show()
```



Decide the final size first; then fonts and margins follow. The same rule in R (Part B).

Python in one slide

- **Text:** strings, len, .count, .replace, .find, slicing [a:b] (0-based, end excluded).
- **Files:** open(path) → lines; rstrip(); open(path, "w") to write.
- **Collections:** list [a, b] (ordered), dict {key: value} (look-up).
- **Control:** if / elif / else; for item in collection; blocks by indentation.
- **Functions & patterns:** def ... return; re.search / re.findall.
- **Libraries:** pandas, matplotlib, Biopython.

This is Jones's chapters 2–8 in one hour. The rest is practice.

Practice (Jones-style exercises, on the site)

- 1 Write `reverse_complement(dna)` and test it on 'ATGC' → 'GCAT'. What happens with an N?
- 2 Count each 3-mer in the COMT CDS with a dictionary; print the three most common.
- 3 Find all EcoRI (GAATTC) and BamHI (GGATCC) sites in `COMT_gene.fa` with `re.finditer`; report 1-based positions.
- 4 Read `reads_5k.fastq`; keep reads whose mean quality is ≥ 30 ; write them to a new FASTQ.
- 5 In pandas: which gene type has the largest median length on chr22? Plot a boxplot of $\log_{10}$ length by type.

Part B

R, by translation

Source structure: Wickham, Çetinkaya-Rundel & Grolemund, R for Data Science 2e (r4ds.hadley.nz) · Grolemund, Hands-On Programming with R · Software Carpentry, R for Reproducible Scientific Analysis

Why a second language? (R4DS, Welcome)

- R was designed by statisticians for data analysis: import → tidy → transform → visualize → model → communicate (the R4DS cycle).
- A t-test, a linear model and a plot are one line each and built into the language; no import needed.
- Bioconductor (2001): DESeq2, edgeR, limma, Seurat — the methods of genomics are published here first, with vignettes you can now read.
- Only four things change from Python: <- for =; counting from 1; vectors instead of loops; braces instead of indentation.

Learn R as a dialect of what you already know.

B1 • First lines (HOPR ch. 1)

R

```
3 + 4  
# → [1] 7
```

```
gene <- "COMT"           # <- is assignment (= also works)  
n_aa <- 271  
cat(gene, "has", n_aa, "amino acids\n")  
# → COMT has 271 amino acids
```

```
?nchar                   # help on a function  
gene                     # typing a name prints it
```

[1] means 'element 1 of a vector': in R even a single number is a vector of length 1.

B2 • Vectors: R's unit of thought (HOPR ch. 2; R4DS ch. 12)

R

```
lengths <- c(816, 1260, 28204)      # c() combines values
length(lengths)                     # → 3
lengths[1]                          # → 816      the FIRST element is 1
lengths[2:3]                        # → 1260 28204

lengths / 1000                      # → 0.816 1.260 28.204  the whole vector at once
lengths > 1000                      # → FALSE TRUE TRUE    a logical vector
lengths[lengths > 1000]             # → 1260 28204        filter by condition
sum(lengths); mean(lengths); max(lengths); sort(lengths)
seq(1, 10, by = 3)                  # → 1 4 7 10
rep("A", 3)                         # → "A" "A" "A"
```

Where Python loops over a list, R operates on the whole vector in one line. Most loops you would write in R are unnecessary.

Translate

YOUR TURN Python: `[x / 1000 for x in lengths if x > 1000]` — in R:

- A. `for (x in lengths) if (x > 1000) x / 1000`
- B. `lengths[lengths > 1000] / 1000`
- C. `lengths / 1000 > 1000`

ANSWER B. Filter with a logical vector inside `[ ]`, then divide the whole result.

B3 • Strings and DNA in R (R4DS ch. 14)

R

```
dna <- "ATGCCGGAGGCC"
nchar(dna)                # → 12      (length(dna) is 1 – one string!)
substr(dna, 1, 3)         # → "ATG"    positions 1–3, both included
substr(dna, 4, 6)         # → "CCG"
toupper(dna); tolower(dna)
gsub("T", "U", dna)       # → "AUGCCGGAGGCC"  replace all
grepl("GAATTC", dna)      # → FALSE    is the pattern there?
regexpr("CCG", dna)       # → 4        where (1-based)
paste0(dna, "TGA")        # concatenate

bases <- strsplit(dna, "")[[1]]      # split into a vector of letters
sum(bases == "G")                    # → 5
mean(bases %in% c("G", "C"))        # → 0.75  GC content, no loop
```

Turn the string into a vector of letters (strsplit); then every question is a vector question. Regular expressions work here too (grepl, gsub, regmatches).

B4 • Named vectors and lists: R's dictionaries

R

```
code <- c(ATG = "M", GTG = "V", GAG = "E", TAA = "*")
code["GTG"]           # → "V"
names(code)           # → "ATG" "GTG" "GAG" "TAA"
code[c("ATG", "GTG")] # look up several at once → "M" "V"
```

a list holds mixed things (HOPR ch. 5)

```
gene <- list(name = "COMT", chrom = "chr22", cds_length = 816)
gene$name           # → "COMT"
gene[["cds_length"]] # → 816
```

the full genetic code, as in Python

```
b <- c("T", "C", "A", "G")
aas <- strsplit(paste0("FFLLSSSSYY**CC*WLLLLPPPPHHQQRRRR",
                       "IIIMTTTTNNKKSSRRVVVVAAAADDEEGGGG"), "")[[1]]
```

```
codons <- character(0)
```

```
for (x in b) for (y in b) for (z in b) codons <- c(codons, paste0(x, y, z))
```

```
codon_table <- setNames(aas, codons)
```

A named vector looks up a whole vector of keys at once. A list is the general container (a data frame is a list of equal-length vectors).

B5 • if, for, function (HOPR ch. 2, R4DS ch. 25)

R

```
q <- 35
if (q >= 30) {
  cat("good base\n")
} else if (q >= 20) {
  cat("acceptable\n")
} else {
  cat("discard\n")
}

for (base in c("A", "T", "G")) cat(base, "\n")

gc_content <- function(dna) {
  bases <- strsplit(dna, "")[[1]]
  mean(bases %in% c("G", "C"))      # the last value is returned
}
```

```
gc_content("ATGCCGGAGGCC")      # → 0.75
```

Condition in (), block in { }. `apply(vector, function)` replaces most loops. `apply` to each

Read a FASTA and translate COMT in R

R

```
read_fasta <- function(path) {  
  lines <- readLines(path)  
  paste(lines[!startsWith(lines, ">")], collapse = "")  
}  
cds <- read_fasta("data/COMT_cds.fa")  
nchar(cds) # → 816  
  
translate <- function(dna) {  
  n <- nchar(dna) %/% 3  
  starts <- seq(1, by = 3, length.out = n) # 1, 4, 7, ...  
  paste(codon_table[substring(dna, starts, starts + 2)], collapse = "")  
}  
protein <- translate(cds)  
substr(protein, 1, 10) # → "MPEAPPLLLA"  
i <- (158 - 1) * 3 + 1 # codon 158 → position 472  
substr(cds, i, i + 2) # → "GTG"  
mutant <- cds; substr(mutant, i, i) <- "A"  
substr(translate(mutant), 158, 158) # → "M"
```

Same result as Python; note the vector move — all codon start positions computed at once, all looked up at once.

B6 • Data frames (R4DS ch. 3; HOPR ch. 5)

R

```
genes <- read.delim("data/chr22_genes.tsv")      # tab-separated
dim(genes)                                     # → 1747 7
str(genes)                                     # structure: column names and types
head(genes, 3)
genes$gene_type[1:3]                          # $ picks a column (a vector)
table(genes$gene_type)                        # counts per category

pc <- genes[genes$gene_type == "protein_coding", ] # [rows, cols]
nrow(pc)                                     # → 447
median(pc$length_bp)                         # → 23780
head(pc[order(-pc$length_bp), c("gene_name", "length_bp")], 3)
# → LARGE1 760660 · TTC28 701827 · SYN3 550562
```

df[rows, columns] — the comma matters. Always run `str()` after reading a file: a numeric column read as text is the most common silent error.

Group, summarize, join; write results

R

```
tapply(genes$length_bp, genes$gene_type, median) # per-group summary
aggregate(length_bp ~ gene_type, data = genes, FUN = median) # formula form

expr <- read.delim("data/chr22_expression.tsv")
m <- merge(genes, expr[, c("gene_id", "TPM")], by = "gene_id") # join

on <- m[m$TPM > 1, ]
head(sort(table(on$gene_type), decreasing = TRUE), 3)
head(expr[order(-expr$TPM), c("gene_name", "TPM")], 3)
# → IGLC2 · IGLV2-23 · ACTBP15 (immunoglobulins: a B-cell sample)

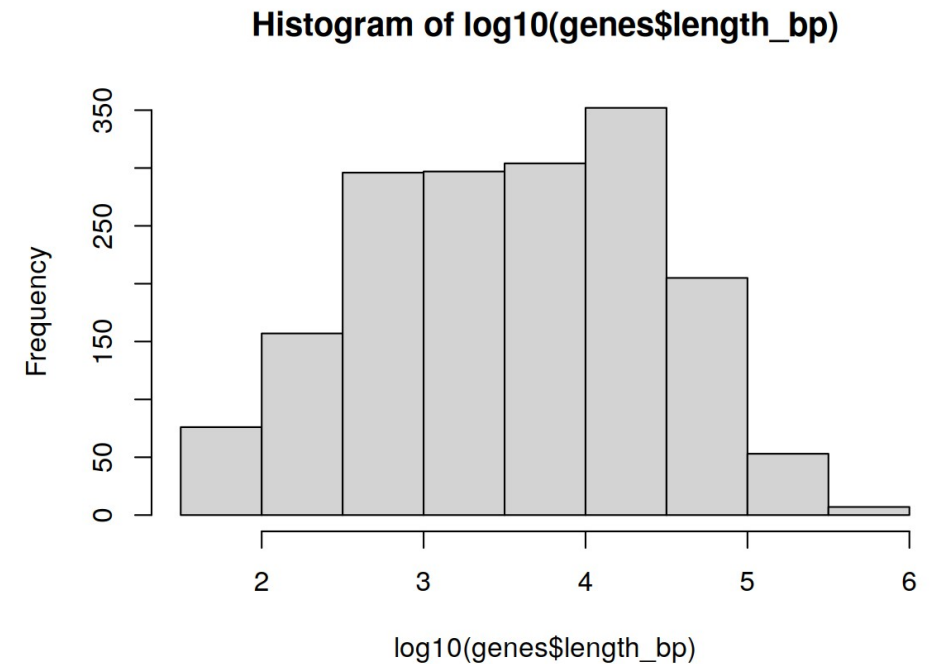
write.table(on, "expressed_genes.tsv", sep = "\t",
            quote = FALSE, row.names = FALSE)
```

pandas groupby → tapply/aggregate; merge → merge; sort_values → order. (The tidyverse — dplyr's filter/arrange/group_by/summarize — is the same five verbs with different names.)

B7 • Plotting is built in — defaults are for the screen

R

```
hist(log10(genes$length_bp))
```

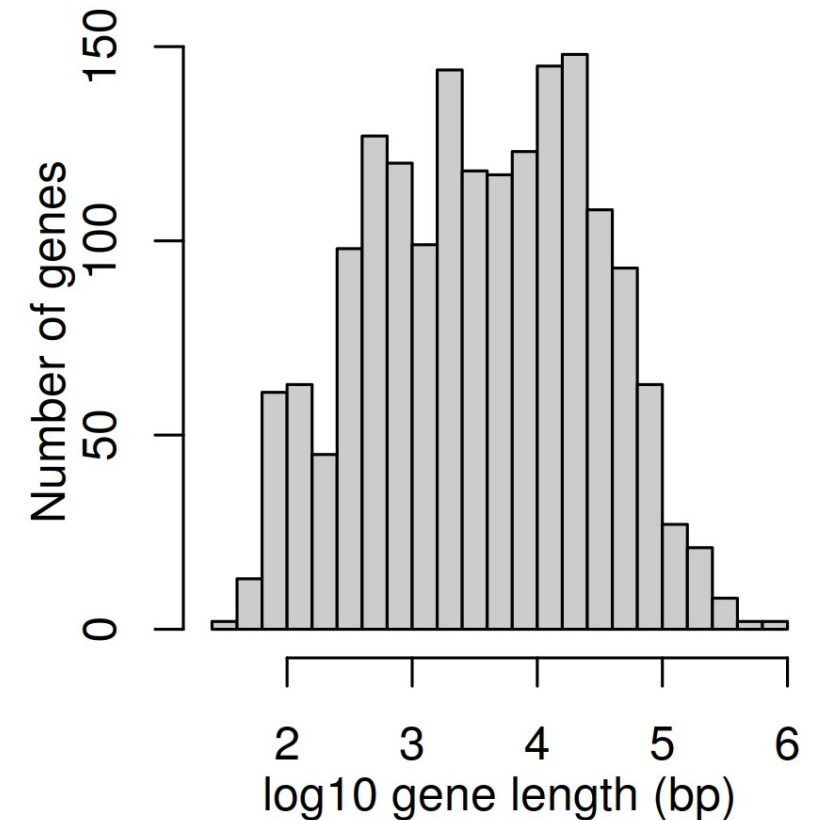


One line and it works. Shrunk to a journal column, the labels vanish — the CADSA workshop lesson.

The publication recipe: think in inches first (CADSA workshop)

R

```
png("fig1.png", width = 3.2, height = 3.2, units = "in",
    res = 300)
par(mar = c(4, 4, 1, 1))          # margins:
bottom, left, top, right
hist(log10(genes$length_bp), breaks = 30, col = "grey80",
     main = "", xlab = "", ylab = "", axes = FALSE) #
draw nothing automatic
axis(1, line = 0.1); axis(2, line = 0.1) # then add axes
yourself
title(xlab = "log10 gene length (bp)", line = 2)
title(ylab = "Number of genes", line = 2)
dev.off()
```

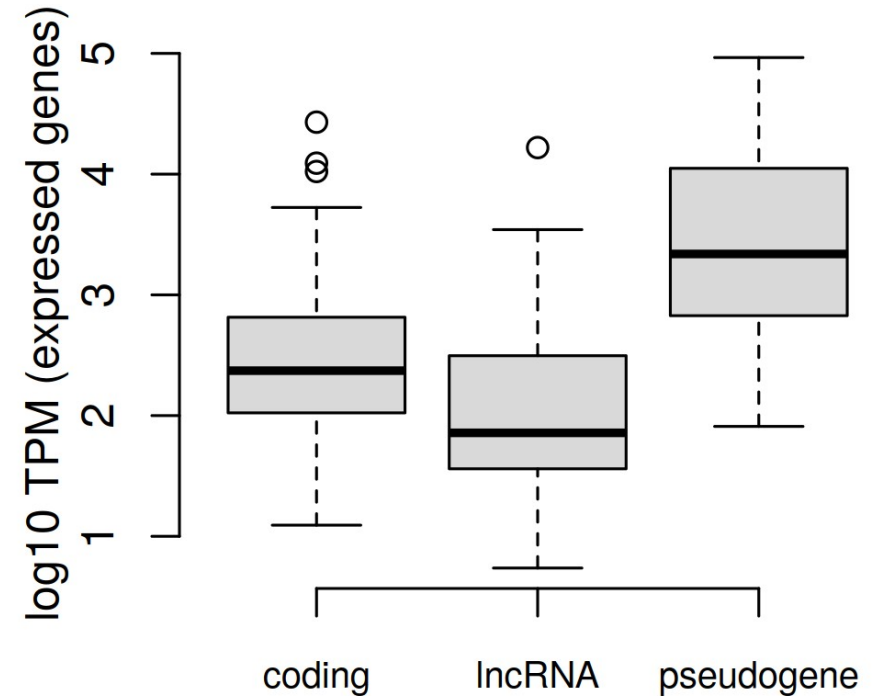


Canvas size → margins → suppress defaults → add axes and labels with sizes you control. Same recipe for `plot()`, `boxplot()`, `barplot()`; `layout()` for panels.

Boxplot by group: the formula $y \sim \text{group}$

R

```
m <- m[m$gene_type %in% c("protein_coding", "lncRNA",  
                          "processed_pseudogene") & m$TPM  
> 0, ]  
boxplot(log10(TPM) ~ gene_type, data = m,  
        ylab = "log10 TPM (expressed genes)")
```



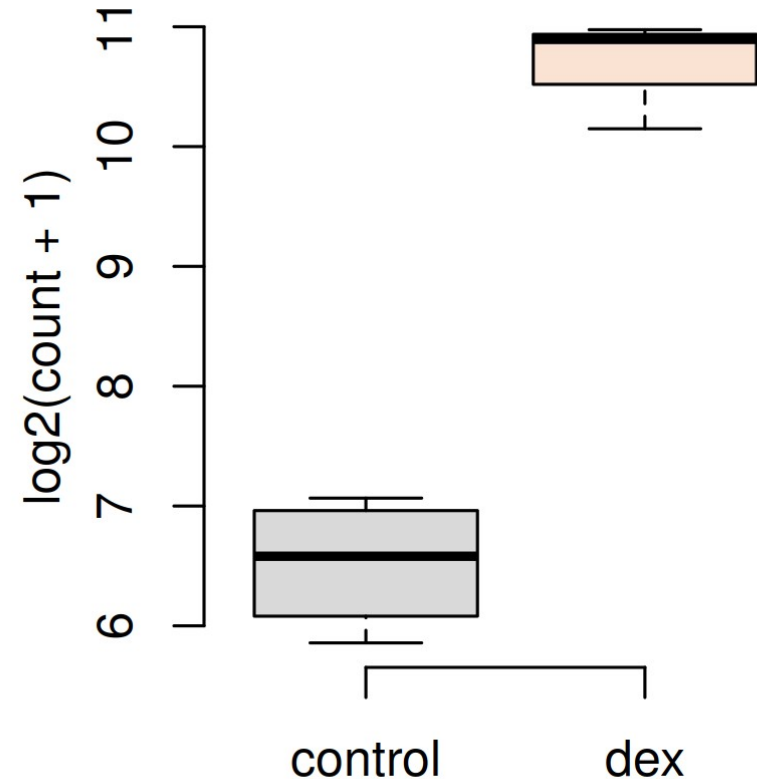
$y \sim \text{group}$ is R's way of saying 'y by group'. The same formula drives `t.test`, `lm`, `glm` — and `DESeq2`'s design (tomorrow).

B8 • Statistics is one line: t.test on real counts

R

```
cts <- read.delim("data/airway_counts.tsv", row.names = 1)
meta <- read.delim("data/airway_metadata.tsv")
logc <- log2(cts + 1)
ctrl <- meta$id[meta$dex == "control"]
trt <- meta$id[meta$dex == "treated"]

g <- "ENSG00000152583" # SPARCL1
a <- unlist(logc[g, ctrl]); b <- unlist(logc[g, trt])
boxplot(list(control = a, dex = b))
t.test(b, a)
# → t = 12.5, df = 6, p-value = 3e-05
# → 95% CI of the difference: 3.3 to 4.9 (log2 units ≈ 16-fold)
```



Four donors ± dexamethasone (Himes 2014). R prints the test, the CI and the means in one call.
Tomorrow: why this is still the wrong tool for 19,271 genes.

Missing values and factors

R

```
x <- c(816, NA, 28204)
mean(x)                      # → NA          one NA poisons the mean
mean(x, na.rm = TRUE)       # → 14510
is.na(x)                     # → FALSE TRUE FALSE
sum(is.na(genes$gene_name)) # how many missing names?

# a factor is a categorical variable with fixed levels
dex <- factor(c("control", "treated", "control", "treated"))
levels(dex)                  # → "control" "treated"
table(dex)
dex <- relevel(dex, ref = "control") # the baseline for models
```

NA is R's 'missing'; most functions refuse to guess unless you say `na.rm = TRUE`. Factors set the reference level for `t.test`/`lm`/`DESeq2` designs.

The apply family: a loop in one call

R

```
# quality per read from a FASTQ, no explicit loop
lines <- readLines("data/reads_5k.fastq")
quals <- lines[seq(4, length(lines), by = 4)]

Q <- t(sapply(quals, function(s) utf8ToInt(s) - 33)) # 5000 x 100 matrix
dim(Q) # → 5000 100
mean_q <- colMeans(Q) # mean per position
round(mean_q[seq(1, 100, 20)], 1) # → 33.4 36.8 36.5 36.1 35.7

read_mean <- rowMeans(Q) # mean per read
sum(read_mean >= 30) # reads passing a Q30 filter
```

Where Python nested two loops, R builds a matrix and takes column means.
`sapply/apply/rowMeans/colMeans` are the idiom.

Predict (data frames)

YOUR TURN `genes[genes$strand == "-" & genes$gene_type == "protein_coding", "gene_name"]` returns...

- A. a data frame with one column
- B. a character vector of gene names
- C. the number of minus-strand protein-coding genes
- D. an error: `&` is not allowed in `[ ]`

ANSWER B. Selecting a single column with `[rows, "col"]` drops to a vector. Wrap in `length( )` for C; use `drop = FALSE` to keep a data frame.

A complete analysis in 15 lines (reproducible script)

R

```
# analysis.R – run with: Rscript analysis.R
genes <- read.delim("data/chr22_genes.tsv")
expr  <- read.delim("data/chr22_expression.tsv")
m <- merge(genes, expr[, c("gene_id", "TPM")], by = "gene_id")
m$expressed <- m$TPM > 1
res <- aggregate(expressed ~ gene_type, data = m, FUN = mean)
res <- res[order(-res$expressed), ]
write.table(res, "expressed_fraction_by_type.tsv", sep = "\t",
            quote = FALSE, row.names = FALSE)
pdf("fig_length_by_type.pdf", width = 3.2, height = 3.2)
par(mar = c(5, 4, 1, 1))
boxplot(log10(length_bp) ~ gene_type, data = m[m$gene_type %in%
        c("protein_coding", "lncRNA"), ], las = 2, ylab = "log10 length")
dev.off()
sessionInfo()
```

Input files → transformations → written outputs → sessionInfo(). Anyone (including you in six months) can re-run it. This is what 'reproducible' means.

Packages, help, and notebooks

- **CRAN:** `install.packages("ggplot2"); library(ggplot2)`.
- **Bioconductor:** `install.packages("BiocManager"); BiocManager::install("DESeq2")`.
- **Help:** `?t.test`; `vignette("DESeq2")` — a vignette is a worked tutorial shipped with the package.
- **Notebooks:** R Markdown / Quarto (.qmd) keep code, output and text together (R4DS ch. 28). This course site is one.
- **Versions:** `sessionInfo()` at the end of every analysis — reviewers will ask.

R in one slide

- **Vectors:** `c()`, 1-based, whole-vector arithmetic, `v[condition]`.
- **Strings:** `nchar`, `substr`, `strsplit`, `paste`, `gsub/grepl`; named vector = dictionary; list = general container.
- **Control:** `if ( ) { }`, `for ( ) { }`, `function( ) { }`, `sapply`.
- **Tables:** `read.delim`, `str`, `df$col`, `df[rows, cols]`, `order`, `table`, `tapply`, `merge`, `write.table`.
- **Plots & stats:** `hist/plot/boxplot` with the inches recipe; `t.test`, `lm`, `y ~ group`.
- **Packages:** CRAN, Bioconductor, vignettes.

You can now read a DESeq2 vignette. Tomorrow you will understand what it computes.

Practice (on the site, in R)

- 1 Rewrite `reverse_complement()` in R using a named vector and `rev()`; test on 'ATGC'.
- 2 Count 3-mers of the COMT CDS with `table(substring(...))`; show the top three.
- 3 With `gregexpr("GAATTC", ...)` list all EcoRI sites in `COMT_gene.fa` (1-based).
- 4 Read `reads_5k.fastq` with `readLines`; compute mean quality per read; keep reads ≥ 30 .
- 5 Boxplot of $\log_{10}$ gene length by gene type, exported at 3.2×3.2 inches.

Part C

Both languages

When to use which, the bugs that bite everyone, and how to use ChatGPT without being used by it.

The Rosetta stone (full table on the site)

Task	Python	R
assign	<code>x = 5</code>	<code>x <- 5</code>
first letter / element	<code>s[0]</code>	<code>substr(s, 1, 1) / v[1]</code>
length	<code>len(s)</code>	<code>nchar(s) / length(v)</code>
count a letter	<code>s.count("G")</code>	<code>sum(strsplit(s, "")[[1]] == "G")</code>
pattern	<code>re.search(p, s)</code>	<code>grepl(p, s)</code>
look-up table	<code>{"ATG": "M"}</code>	<code>c(ATG = "M")</code>
filter a table	<code>df[df["x"] > 1]</code>	<code>df[df\$x > 1,]</code>
group + summarize	<code>df.groupby("g")["x"].median()</code>	<code>tapply(df\$x, df\$g, median)</code>
t-test	<code>scipy.stats.ttest_ind(a, b)</code>	<code>t.test(a, b)</code>
install	<code>pip install biopython</code>	<code>BiocManager::install("DESeq2")</code>

Which language for which step?

The data are...	Think...	Reach for...
Sequences, reads, files, a tool to glue	text	Python (or the shell)
One row per gene, columns of numbers	table	either; R if statistics come next
Counts with replicates and a hypothesis	table + uncertainty	R: DESeq2, edgeR, limma
Images, deep learning, single cell at scale	arrays	Python: PyTorch, scanpy
A figure for the paper	inches	base R (or matplotlib) with the recipe

The five bugs that bit real papers

Bug	What happens	Defence
0- vs 1-based	Off by one base: wrong codon, wrong fragment	Convert once on paper; test on a tiny example
Strand	A minus-strand pattern never matches	Reverse-complement before searching
Reference build	GRCh37 positions in a GRCh38 file	Write the build in every file name
Excel gene names	SEPT7 → 7-Sep, MARCH1 → 1-Mar (≈20 % of supplementary tables; genes renamed 2020)	Never open a gene list in Excel
Silent type coercion	A numeric column read as text; p-values sorted as strings	<code>str(df) / df.dtypes</code> after every read

None of these crash. All of them produce a confident, wrong table.

ChatGPT writes code. You verify it.

- Ask with TRACE (Task, Role, Audience, Context, Example), then read every line and ask it to explain any you cannot.
- **Test on a known case:** reverse complement of ATGC is GCAT; COMT codon 158 is GTG; the protein starts with M and has 271 aa.
- Copilot-style autocomplete is off in the lab: the exercises are exactly the lines it would type for you.
- Lab 1 ends with a ChatGPT function that fails two of three tests. Your job is to find out why.

Where to go next

- **Python:** Jones, Python for Biologists (free tutorial at pythonforbiologists.com); Software Carpentry, Programming with Python; Biopython Tutorial.
- **R:** Wickham et al., R for Data Science 2e (r4ds.hadley.nz, free); Grolemund, Hands-On Programming with R (free); Software Carpentry, R for Reproducible Scientific Analysis.
- **Genomics in R:** Holmes & Huber, Modern Statistics for Modern Biology (free; tomorrow's source); Bioconductor vignettes.
- **Practice:** Rosalind (rosalind.info) — bioinformatics problems in increasing difficulty, in any language.

Lab 1 on Wednesday (1.5 h, browser only)

- Ex 1 Hello, two languages • Ex 2 COMT: GC, reverse complement, translate, rs4680 • Ex 3 FASTQ quality
- Ex 4 Tables: genes, expression, mismatches • Ex 5 The t-test and the 19,271-tests trap • Ex 6 Catch ChatGPT's bug
- Every exercise in both languages with hints and solutions on the site; the handout has the same exercises with blanks.

Questions?

jinrui.xu@howard.edu

Course site: notes, Rosetta stone, Lab 1, data, local setup for Windows and Mac.